

General Project Report — Building AI Agent

Introduction

Building AI Agent is a progressive educational project for building an AI agent from scratch. The project begins with a very small notebook-based agent and evolves step by step through routing, parsing, execution, memory, tool metadata, local language-model integration, and finally multi-step execution.

The project does not begin with a framework and does not begin with a cloud inference API. Instead, it gradually introduces each architectural responsibility so that every design choice can be understood clearly.

Project Goal

The goal of the project is to build an agent capable of:

- receiving a user request;
- understanding the user intent;
- selecting the correct tool;
- extracting the required arguments;
- validating and executing the selected tool;
- storing the interaction state;
- handling small multi-step requests;
- evolving through a clear modular architecture.

Development Philosophy

The project follows one simple rule:

one version, one concept

Each version introduces only one important architectural idea. This keeps the learning process controlled and makes the evolution easy to follow.

```
simple agent
↓
separated execution
↓
generic parsing
↓
planning
↓
managed memory
↓
tool registry
↓
LLM routing
↓
LLM parsing
↓
multi-step execution
```

General Architecture

Across the project, the agent is designed as a pipeline of specialized components. In the final implemented version the simplified flow is:

```
User Request
↓
Planner
↓
LLM Router
↓
LLM Parser
↓
Executor
↓
Memory Manager
↓
Memory / Output
```

The Tool Registry supports this flow by storing tool functions, parameters, descriptions, and synonyms. The Registry is shared by the Router, Parser, Planner, and Executor.

Core Components

Router

The Router decides which tool should be used. In early versions it is rule-based; in Version 7 it becomes an LLM Router that uses a small local language model and the tool descriptions stored in the Tool Registry.

Parser

The Parser converts natural language into structured arguments. In Version 3 it becomes generic and reusable. In Version 8 it becomes an LLM Parser, capable of extracting values semantically from more natural requests.

Executor

The Executor validates the arguments and runs the selected function. It exists to keep execution logic separate from the Agent coordinator.

Planner

The Planner creates an execution plan. In Version 4 it introduces explicit planning before execution. In Version 9 it evolves into a simple multi-step planner that can produce an ordered list of steps.

Memory Manager

Memory stores the state of each interaction. In Version 5 memory management is extracted into dedicated functions. In later versions memory keeps the entire multi-step state, including plans and executed steps.

Tool Registry

The Tool Registry is introduced in Version 6 to centralize tool metadata. It stores the function, parameters, description, and synonyms for each tool, and becomes the shared source of truth for the rest of the system.

Implemented Roadmap

V1 → Agent Base
V2 → Executor
V3 → Generic Parser
V4 → Planner
V5 → Memory Manager
V6 → Tool Registry
V7 → LLM Router
V8 → LLM Parser
V9 → Multi-step Agent

Version 9 is the final implemented version of the current project roadmap.

Version-by-Version Summary

V1 — Agent Base

Introduces the minimal agent structure with routing, parsing, tools, and a simple state.

V2 — Executor

Separates validation and execution from the Agent so that the Agent becomes a coordinator.

V3 — Generic Parser

Replaces rigid parsing logic with a reusable component that can extract arguments for multiple tools.

V4 — Planner

Adds an explicit plan that explains which tool should be used, why, and which arguments are required.

V5 — Memory Manager

Introduces dedicated memory functions such as saving, reading, clearing, and retrieving the last interaction.

V6 — Tool Registry

Centralizes tool metadata and removes the need for separate structures spread across the notebook.

V7 — LLM Router

Introduces semantic tool selection through a local language model while keeping the rest of the architecture stable.

V8 — LLM Parser

Introduces semantic argument extraction through the same local language model and returns structured JSON-style arguments.

V9 — Multi-Step Agent

Introduces sequential execution. The planner creates multiple steps, the Agent executes them one at a time, and the result of one step can be reused in the next.

Educational Value

The project teaches more than coding. It teaches how to think about agent architecture through separation of responsibilities, modular design, incremental development, controlled use of local language models, tool-based execution, memory management, and planning.

Final Architecture and Final Scope

The final implemented scope of the project is a notebook-based multi-step agent. It is intentionally simple and educational. The system can:

- route semantically with a local LLM;
- parse arguments semantically with the same local LLM;
- store tool metadata in a Tool Registry;
- create a small execution plan;
- execute multiple ordered steps;
- pass the result of one step to the next;
- save the full interaction state in memory.

Several limitations are intentionally kept for clarity. The multi-step logic depends on simple splitting with the word "then". The project remains notebook-based, the registry is in-memory, and the memory is a simple list. These choices are consistent with the educational purpose of the work.

Conclusion

Building AI Agent is a structured learning project that demonstrates how an AI agent can evolve from a simple rule-based notebook into a modular system with a Tool Registry, local language-model components, and multi-step execution.

Its main strength is controlled evolution. Each version adds one new concept and improves the architecture without creating unnecessary complexity. The project now concludes at Version 9, where the final implemented result is a clear and functional multi-step agent built entirely in notebooks.